Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Лабораторная работа №1

«Основные конструкции языка Python»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-32Б _____ (Сердюков А.С.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цель и задание

Разработать три консольные программы для решения биквадратного уравнения и сравнить процедурную, объектно-ориентированную и функциональную парадигмы.

Для уравнения $A \cdot x^4 + B \cdot x^2 + C = 0$ вводятся коэффициенты из командной строки или с клавиатуры. При ошибке ввода соответствующий коэффициент запрашивается повторно. Программа выводит дискриминант и действительные корни.

Описание решения

Замена $y = x^2$ сводит задачу к квадратному уравнению. Из его неотрицательных корней получаются $x = \pm\sqrt{y}$. Повторы удаляются, корни сортируются. Отдельно обработаны $A = 0$, $B = 0$ и тождество $0 = 0$. Для двух различных корней используется формула через q, уменьшающая потерю точности; коэффициенты предварительно нормируются.

Процедурная версия организована функцией solve. В объектной версии коэффициенты и метод решения принадлежат BiquadraticEquation. Функциональная версия использует чистые функции, match/case, filter и map. Общий модуль отвечает за ввод и представление результата. NaN и бесконечности отклоняются. Вычисления ограничены точностью float.

Порядок запуска

Python 3.10 или новее. Из корня репозитория создать виртуальное окружение и установить зависимости командой python -m pip install -r requirements.txt.

```
python lab_01/procedural.py 1 -5 4
python lab_01/object_oriented.py 1 -5 4
python lab_01/functional.py 1 -5 4
```

Результаты выполнения

```
$ python lab_01/procedural.py 1 -5 4
A = 1; B = -5; C = 4
Дискриминант = 9
Действительные корни: -2, -1, 1, 2

$ python lab_01/object_oriented.py 1 -5 4
A = 1; B = -5; C = 4
Дискриминант = 9
Действительные корни: -2, -1, 1, 2

$ python lab_01/functional.py 1 -5 4
A = 1; B = -5; C = 4
Дискриминант = 9
Действительные корни: -2, -1, 1, 2

$ python lab_01/procedural.py 1 0 1
A = 1; B = 0; C = 1
Дискриминант = -4
Действительных корней нет

$ python lab_01/procedural.py 0 0 0
A = 0; B = 0; C = 0
Дискриминант = 0
Решение: любое действительное число

$ python lab_01/procedural.py bad -5 4
Ввод с клавиатуры: 'oops\n1\n'
Некорректный коэффициент A. Введите конечное число.
A = Некорректный коэффициент A. Введите конечное число.
A = A = 1; B = -5; C = 4
Дискриминант = 9
Действительные корни: -2, -1, 1, 2

Итог тестового запуска (полный журнал tests.txt)
$ python -m pytest lab_01 -v --tb=short
31 passed in 0.04s
```

Рисунок 1 — Вывод фактического запуска программы

Экранная форма сформирована из сохранённого stdout. Полные журналы: results/demo.txt и results/tests.txt.

Проверка и выводы

Для этой работы успешно выполнено проверок: 31. Общий набор всех шести работ содержит 61 тест; полный прогон завершился без ошибок. Версии среды и журналы сохранены в validation/ и results/tests.txt.

Три реализации дают одинаковые действительные корни на проверенных наборах коэффициентов. Для уравнения $x^4 - 5x^2 + 4 = 0$ получены корни −2, −1, 1 и 2. Проверены отсутствие корней, повторные и нулевые корни, вырожденные уравнения и повторный ввод.

Источник задания

https://github.com/ugapanyuk/BKIT_2022/wiki/lab_python_intro

Текст программы

```
# Файл common.py

"""Общий ввод и представление результата трёх реализаций."""
import math
import sys
from dataclasses import dataclass


@dataclass(frozen=True)
class Solution:
    roots: tuple[float, ...] = ()
    infinite: bool = False


def read_coefficients(args, input_fn=input, output_fn=print):
    values = []
    for index, name in enumerate("ABC"):
        raw = args[index] if index < len(args) else None
        while True:
            if raw is None:
                raw = input_fn(f"{name} = ")
            try:
                value = float(raw)
                if not math.isfinite(value):
                    raise ValueError
                values.append(value)
                break
            except (ValueError, TypeError):
                output_fn(f"Некорректный коэффициент {name}. Введите конечное
                    число.")
                raw = None
    return tuple(values)


def run(solver):
    try:
        a, b, c = read_coefficients(sys.argv[1:])
        result = solver(a, b, c)
        print(f"A = {a:g}; B = {b:g}; C = {c:g}")
        print(f"Дискриминант = {b * b - 4 * a * c:g}")
        if result.infinite:
            print("Решение: любое действительное число")
        elif result.roots:
            print("Действительные корни:", ", ".join(f"{x:.12g}" for x in
                result.roots))
        else:
            print("Действительных корней нет")
    except (EOFError, KeyboardInterrupt):
        print("\nВвод прерван", file=sys.stderr)
        return 1
    except (ValueError, OverflowError) as error:
        print(f"Ошибка вычисления: {error}", file=sys.stderr)
        return 1
    return 0
```

Текст программы Продолжение

```
# Файл procedural.py

"""Процедурное решение A*x**4 + B*x**2 + C = 0."""
from math import copysign, sqrt
from common import Solution, run


def solve(a, b, c):
    if a == 0:
        if b == 0:
            return Solution(infinite=c == 0)
        squares = (-c / b,)
    else:
        # Масштабирование снижает вероятность переполнения дискриминанта.
        scale = max(abs(a), abs(b), abs(c))
        a, b, c = a / scale, b / scale, c / scale
        d = b * b - 4 * a * c
        if d < 0:
            return Solution()
        if d == 0:
            squares = (-b / (2 * a),)
        else:
            q = -0.5 * (b + copysign(sqrt(d), b))
            squares = (q / a, c / q)
    roots = set()
    for y in squares:
        if y >= 0:
            roots.update((-sqrt(y), sqrt(y)))
    return Solution(tuple(sorted(roots)))


if __name__ == "__main__":
    raise SystemExit(run(solve))


# Файл object_oriented.py

"""Объект хранит коэффициенты и вычисляет корни уравнения."""
from dataclasses import dataclass
from math import copysign, sqrt
from common import Solution, run


@dataclass(frozen=True)
class BiquadraticEquation:
    a: float
    b: float
    c: float

    @property
    def discriminant(self):
        return self.b ** 2 - 4 * self.a * self.c

    def solve(self):
        a, b, c = self.a, self.b, self.c
        if a == 0 and b == 0:
```

Текст программы Продолжение

```
            return Solution(infinite=c == 0)
        if a == 0:
            squares = (-c / b,)
        else:
            scale = max(abs(a), abs(b), abs(c))
            normalized = BiquadraticEquation(a / scale, b / scale, c / scale)
            d = normalized.discriminant
            if d < 0:
                return Solution()
            if d == 0:
                squares = (-normalized.b / (2 * normalized.a),)
            else:
                q = -0.5 * (normalized.b + copysign(sqrt(d), normalized.b))
                squares = (q / normalized.a, normalized.c / q)
        roots = {sign * sqrt(y) for y in squares if y >= 0 for sign in (-1,
            1)}
        return Solution(tuple(sorted(roots)))


if __name__ == "__main__":
    raise SystemExit(run(lambda a, b, c: BiquadraticEquation(a, b,
        c).solve()))


# Файл functional.py

"""Чистые функции и структурное сопоставление с образцом."""
from math import copysign, sqrt
from common import Solution, run


def square_roots(a, b, c):
    match (a, b, c):
        case (0, 0, _):
            return ()
        case (0, _, _):
            return (-c / b,)
        case _:
            scale = max(abs(a), abs(b), abs(c))
            a, b, c = a / scale, b / scale, c / scale
            match b * b - 4 * a * c:
                case d if d < 0:
                    return ()
                case 0:
                    return (-b / (2 * a),)
                case d:
                    q = -0.5 * (b + copysign(sqrt(d), b))
                    return (q / a, c / q)


def solve(a, b, c):
    match (a, b, c):
        case (0, 0, 0):
            return Solution(infinite=True)
        case _:
            values = map(sqrt, filter(lambda y: y >= 0, square_roots(a, b,
```

Текст программы Продолжение

```
                c)))
            return Solution(tuple(sorted({x for r in values for x in (-r,
                r)})))


if __name__ == "__main__":
    raise SystemExit(run(solve))


# Файл test_equations.py

import pytest
from procedural import solve as procedural
from functional import solve as functional
from object_oriented import BiquadraticEquation
from common import read_coefficients


@pytest.mark.parametrize("solver", [procedural, functional,
    lambda a, b, c: BiquadraticEquation(a, b, c).solve()])
@pytest.mark.parametrize("coefficients, roots, infinite", [
    ((1, -5, 4), (-2, -1, 1, 2), False),
    ((1, -2, 1), (-1, 1), False),
    ((1, 0, 1), (), False),
    ((1, 5, 4), (), False),
    ((1, 0, -16), (-2, 2), False),
    ((1, -1, 0), (-1, 0, 1), False),
    ((0, 2, -8), (-2, 2), False),
    ((0, 0, 1), (), False),
    ((0, 0, 0), (), True),
    ((1, 0, 0), (0,), False),
])
def test_real_roots(solver, coefficients, roots, infinite):
    result = solver(*coefficients)
    assert result.roots == pytest.approx(roots)
    assert result.infinite is infinite
    a, b, c = coefficients
    for x in result.roots:
        assert a * x ** 4 + b * x ** 2 + c == pytest.approx(0)


def test_invalid_and_missing_coefficients():
    entries = iter(["bad", "1", "4"])
    messages = []
    assert read_coefficients(["nan", "-5"], lambda _: next(entries),
        messages.append) == (1, -5, 4)
    assert len(messages) == 2
```